

EDAL- TCP/IP&UDP

<https://community.ruggedboard.com>

Transport Layer

Transport Layer (also called Socket Layer)

1. Transport Layer Goals
2. UDP
3. TCP

Application layer

Transport Layer

Network Layer

Data link layer

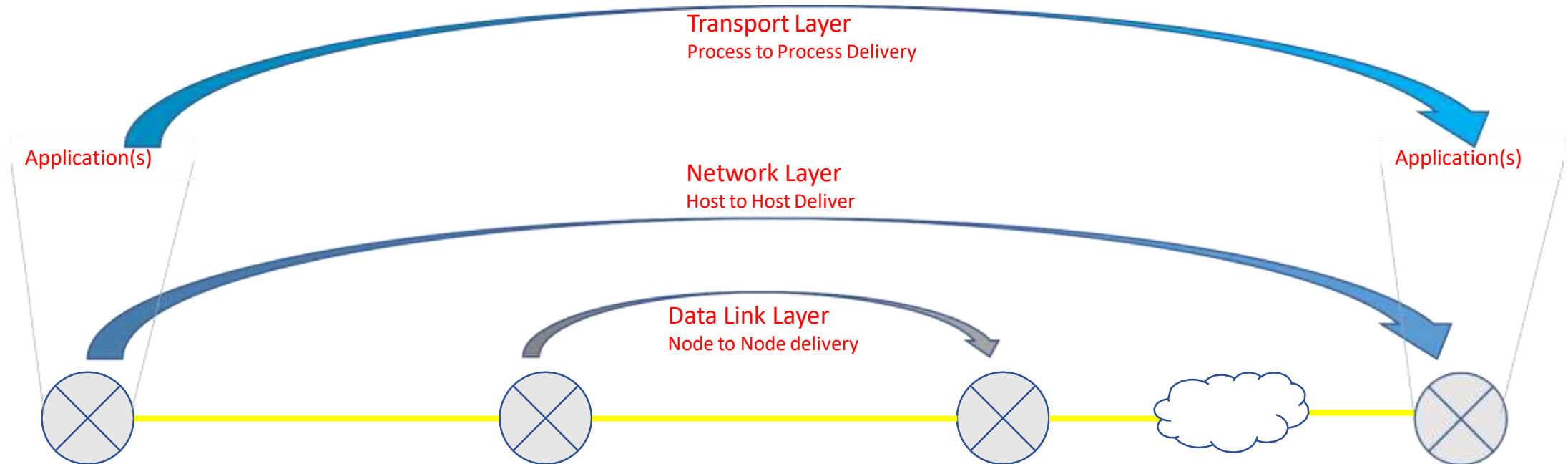
Physical layer

Transport Layer

- **Transport Layer Goals**
 - After the network Layer has verified that packet is not to be forwarded to next hop, and packet is destined to itself, network layer chop off network layer header (IP header) and hands over the remaining packet to upper transport layer
- To achieve its functions, Transport Layer has 3 famous standard protocols :
 - TCP (Transmission control protocol)
 - UDP (User datagram protocol)
 - SCTP (Stream Control Transmission Protocol)
- **Transport Layer Function is:**
 - Identify the correct process/application to deliver the packet (TCP, UDP)
 - Congestion control in the network (TCP only)
 - Reliable delivery of the packets to application (without error) (TCP only)

Transport Layer

- Responsible for process to process delivery
- Not aware of layer above it or below it (This is true for every layer in TCP/IP stack)



Transport Layer

- Two standard and Famous Transport layer Protocols are :
 - UDP (User Datagram Protocol)
 - TCP (Transmission control protocol)

TCP	UDP
If the application needs reliability	Appln do not needs reliability
Slower and complex service	Simpler and fast service
Connection-oriented and reliable	Connectionless and unreliable
Can recover lost packets, detect malformed corrupted packets, react to congestion in the network etc	No such mechanism, a packet lost or corrupted is gone forever
Eg : Sending Email Doc/file	Eg : Audio/Video streaming

Consider TCP and UDP as Twins, both helps achieve the same goal – Data delivery from source PROCESS to destination PROCESS but in a different way.

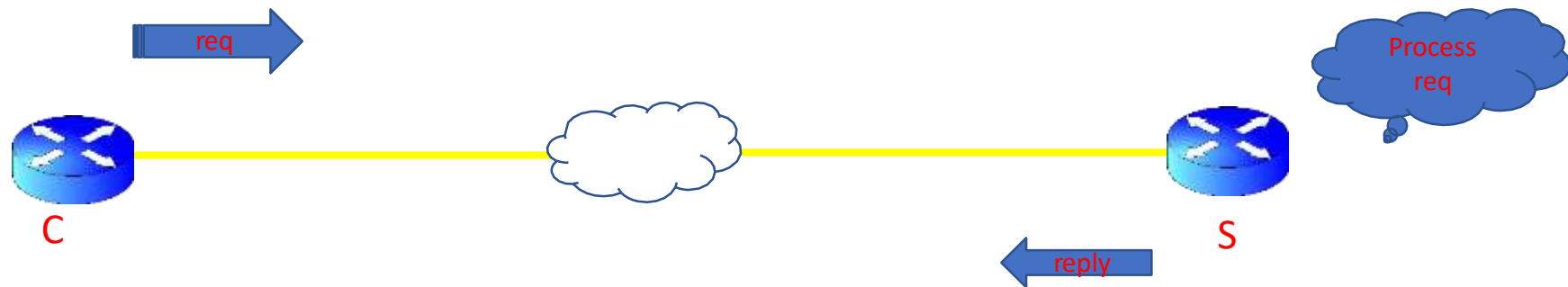
- TCP is a complex protocol

Socket programming

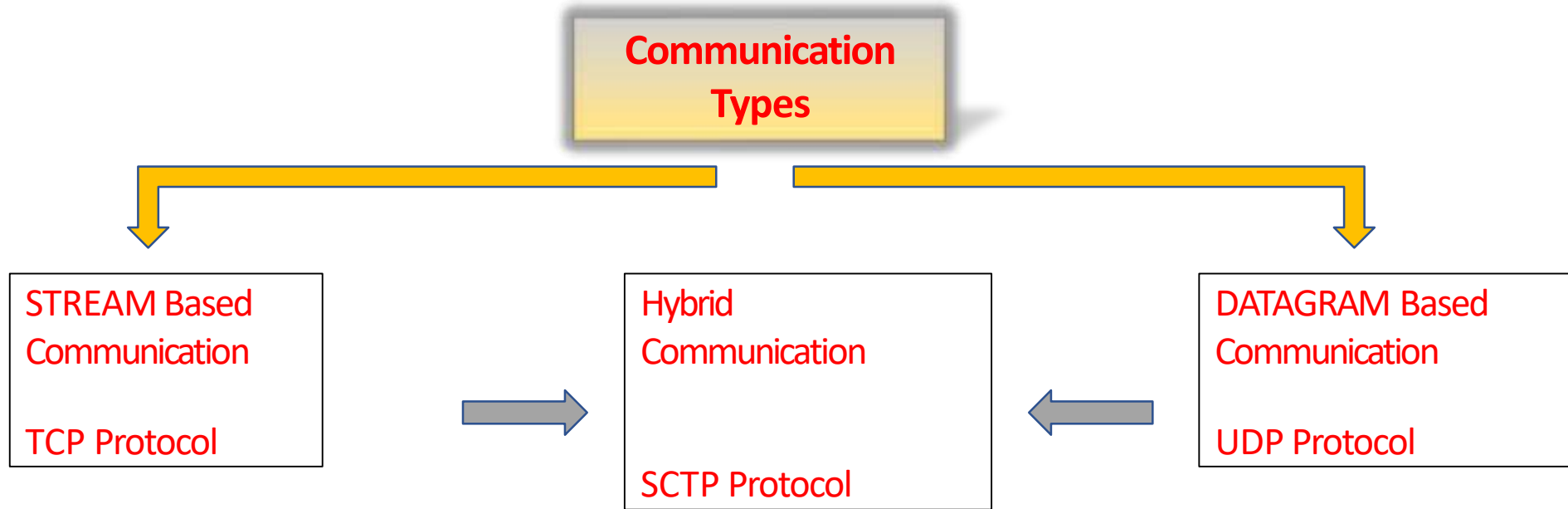
1. Introduction
2. Writing a Client - Server based model in C
3. TCP and UDP based socket programming
4. Multiplexing using Select()
5. TCP server design steps

Socket Programming -> Client and Server based Model

- Server is any machine which receives the request, process it and optionally returns back the result
- Client is any machine which initiates the request
- We will quickly write a simple Client Server Hello world model



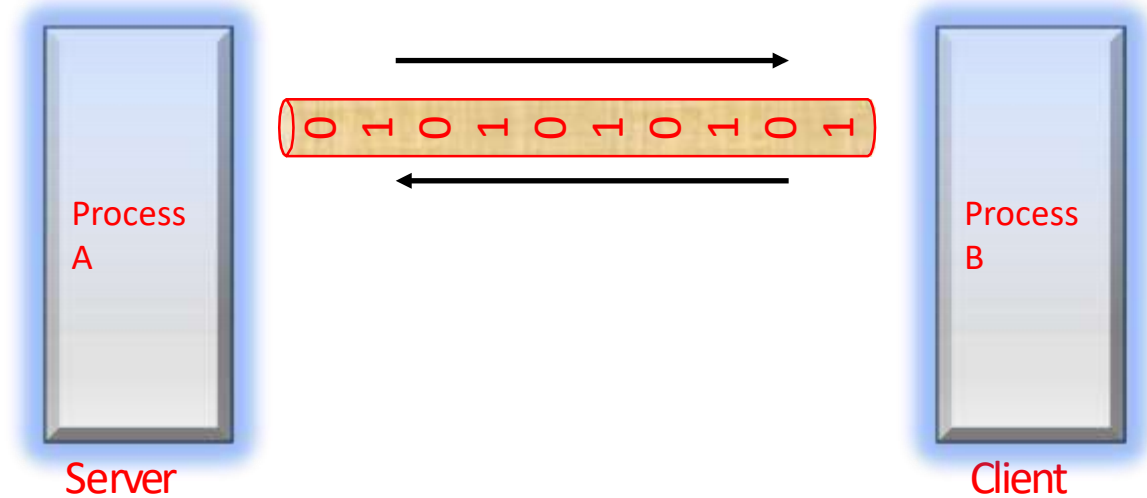
- Server never initiates the communication



- A Communication between two machines can be broadly classified into STREAM or DATAGRAM based Communication
- Which type of communication to setup – depends on end goals
- Each Communication type has its own pros and cons

TCP based Communication

- Connection Oriented : Dedicated connection between A & B
 - Server and Client maintains the internal Data structures to maintain connection session state
 - Communication processes tracks the amount data which is sent, not sent, pending ACK, ACK'ed etc
 - Eg : Downloading software resumes downloading the file from the same state when internet connectivity restores
- Mutual Agreement : 3-Way handshake necessary before any actual data exchange takes place
 - 3-Way handshake is a mechanism using which the communicating processes mutually agrees to participate in bidirectional connection oriented communication
- Reliable : Design to be reliable, recoverable from errors, error correction, deal with congestion
 - A lost Data is re-transmitted by sender, so delivery of all data bytes are guaranteed.
 - If Network is congested, Sender can slow down the rate of data transmission
 - If receiver receives any corrupted data, receiver can ask the sender to retransmit
- STREAM of Bytes: Sender can send continuous stream of bytes of data to recipient, analogous to flow of water through a pipe



TCP based Communication

5. **Sequenced Data** : Data packets arrives in the same sequence to recipient application in which they were transmitted

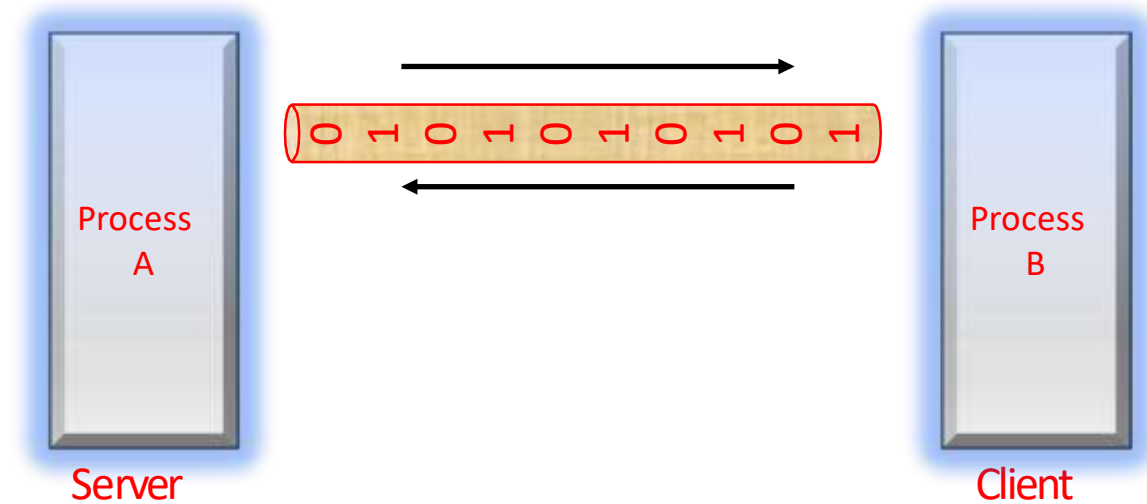
6. **Use-case** : Should be used where receiver can tolerate a time LAG but not packet loss
Or if the info is sensitive towards pkt loss

7. **Example** :

Downloading an Audio Or a Movie Or software *stored on a disk on a server* – *Little Delay is tolerable*

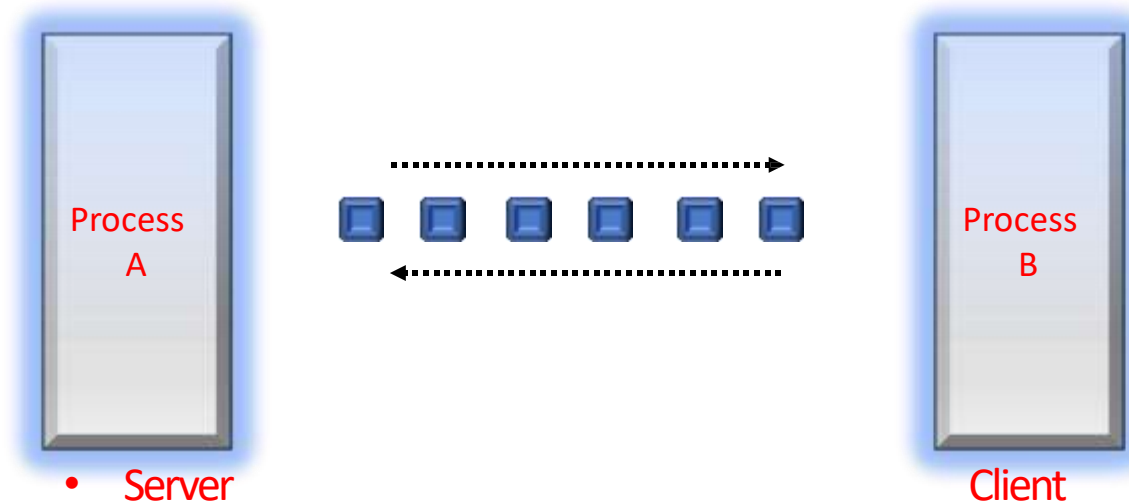
Emails, Chat messages etc – *Sensitive towards packet loss*

Suggested Readings : To Study internal implementation in Detail how STREAM based communication works, you are recommended to study the internal implementation of TCP/IP protocol



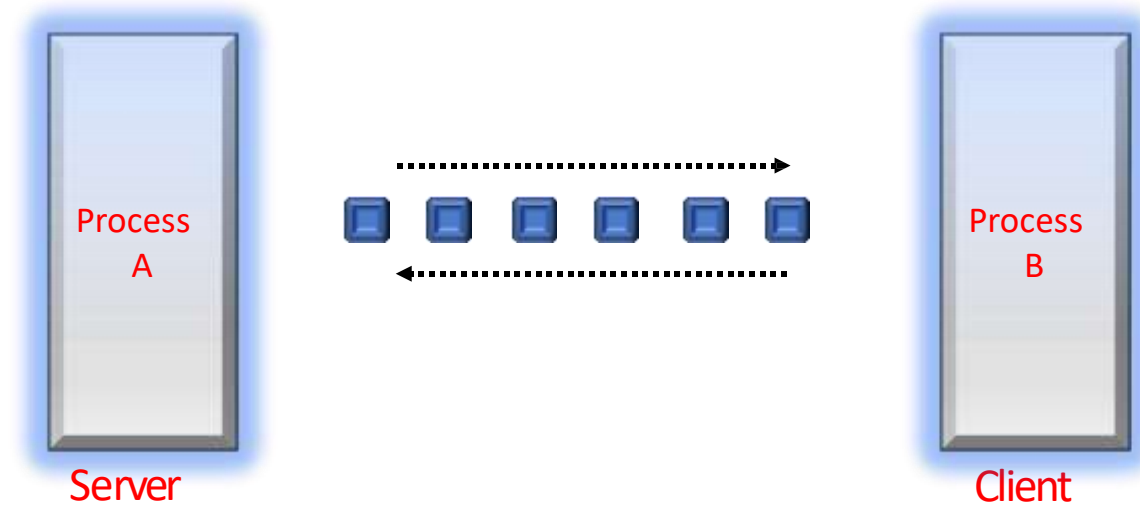
UDP based Communication

- Connection-less : No Connection establishment between A & B
Sending machine do not maintain any connection state Or store the identity of other machine
- Every data transmission instance is a fresh data transmission
- Mutual Agreement : Communicating machine triggers data transmission without any prior mutual agreement Or notification
- Reliable : Not Reliable, not to be used in error prone Networks Or environments
 - A lost Data is lost forever
 - no Recovery mechanism to deal with congested network or corrupted data
- DATAGRAM of Bytes : Data is sent as discrete isolated units of data called Datagrams. Each Datagram is a complete Unit of data by itself
- Sequence : Datagram can arrive out of sequence on recipient machine



UDP based Communication

- 6. Use-case : Should be used where recipient can tolerate a packet loss, but not a time lag
- 7. Example :
 - LIVE Video/Audio conferences
 - You can compromise you Audio/Video Quality, but time lag in LIVE sessions is horrible



TCP/IP & UDP

Scenario	STREAM Or DATAGRAM	Reason
Your finger prints are being collected by scanner and being saved on local Disk	DATAGRAM	No error, no Congestion, No loss. Either of the two could be used
Uploading/downloading Videos on/from Youtube	STREAM	Your Data is going out on open WAN, reliability should be ensured
Downloading ISO image file of Windows 10 from MS Servers	STREAM	Even if one packet is missed, the software may not run at all. Delay in download is tolerable
On Google Hangout or In Team meetings on Skype	DATAGRAM	Time lag is not tolerable, few packet losses are tolerable
Sending Emails	HYBRID	Data loss is not tolerable, Individual units of data is sent
Sending Text messages	HYBRID	Data loss is not tolerable, Individual units of data is sent

Socket Programming -> Select And Accept System Calls

- Linux OS provide two system calls to enable us write efficient Network Applications
 - `Select()`
 - `Accept()`
- In C, it is difficult to write a socket based application without the use of `Select` and `Accept` System calls
- From interview point of view – Understanding these two calls is very important
- Let us first grasp the idea of these two APIs. Having understood these two, remaining socket programming APIs would automatically fall in place

Socket Programming -> Select And Accept System Calls

High level Linux Socket based Communication Design

- Messages (Or requests) send by the client to the server can be categorized into two types :
 - Connection initiation request messages
 - This msg is used by the client to request the server to establish a dedicated connection.
 - Only after the connection has been established, then only client can send Service request messages to server.

And

- Service Request Messages
 - Client can send these msg to server once the connection is fully established.
 - Through these messages, Client requests server to provide a service
- Servers identifies and process both the type of messages very differently

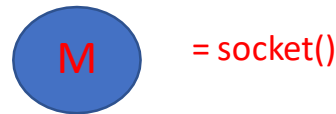
- High level Socket Communication Design

Msg type	Client side API	Server side API
Connection initiation request msgs	connect()	accept()
Service request msgs	sendmsg(), sendto()	recvmsg(), recvfrom()

Socket Programming -> Select And Accept System Calls

- Life Cycle of a Client Server Communication

When Server boots up, it creates a master socket using `socket()`



M is the mother of all Client Handles. M gives birth to all Client handles.

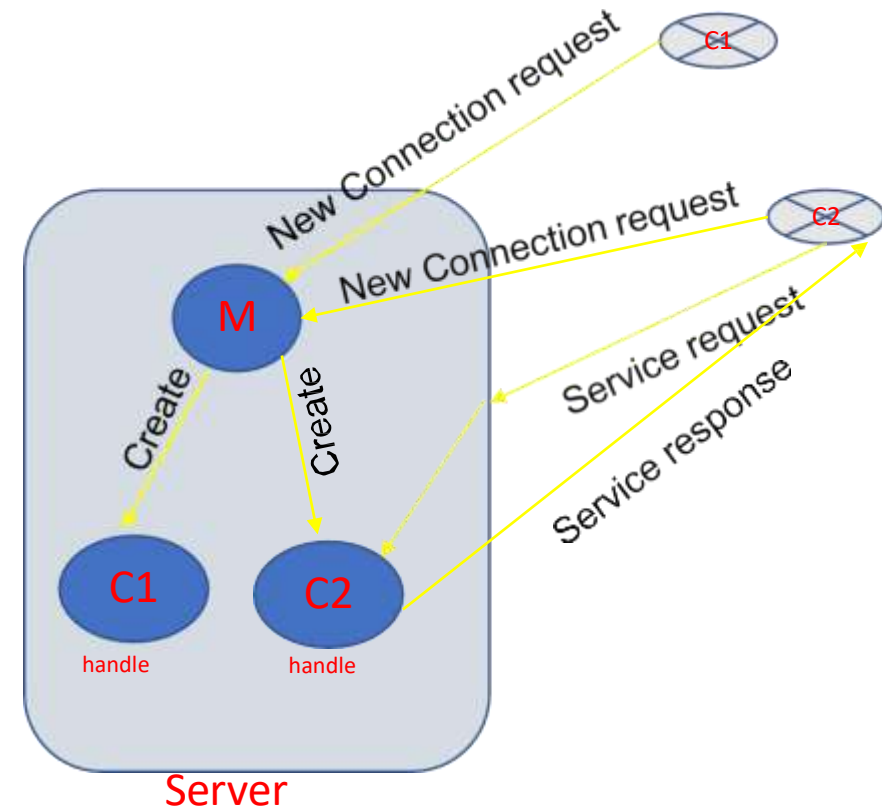
Once Client handles are created for each client, Server carries out Communication with the client using Client handle (and not M).

Server Has to maintain the database of connected client handles

M is only used to create new client handles. M is not used for data exchange with already connected clients.

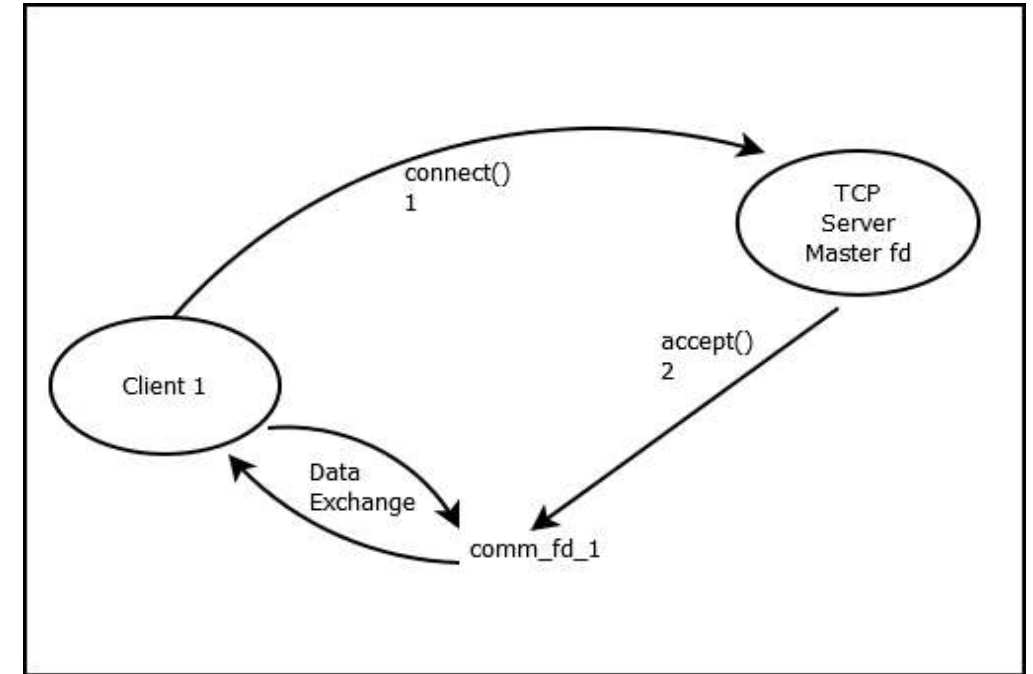
`Accept()` is the system call used on server side to create client handles.

In Linux Terminology, handles are called as “file descriptors” which are integer numbers. Client handles are called “communication file descriptors” and M is called Master socket file descriptor



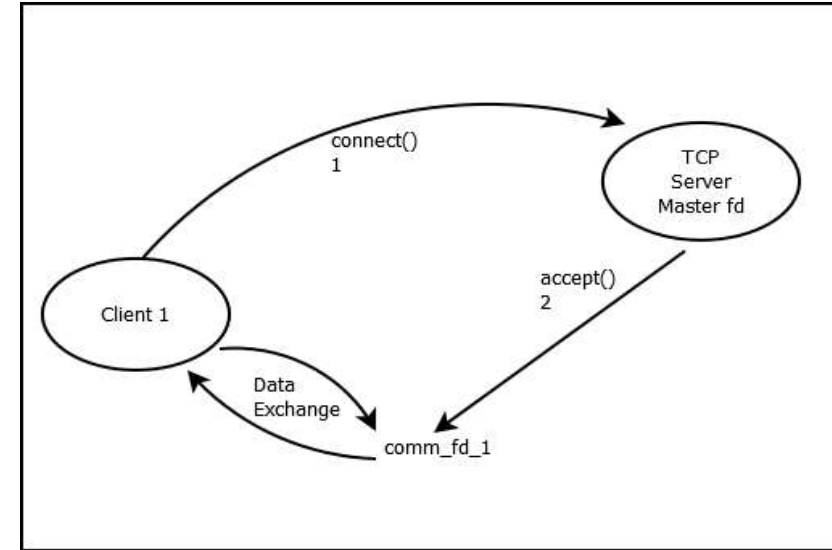
Socket Programming -> Accept() System Call

- Accept()
 - Accept() system call is used by the TCP server (not UDP server) to accept the new client connection request
- Remember, TCP is a connection oriented protocol, meaning – client and server needs to establish dedicated connection first before they can participate in any data exchange activity
- For example, when you login into facebook, your machine sends connection request to Facebook servers. Facebook servers accept your connection request using accept() system call. Now, you can send data to facebook servers, and facebook servers can update your page with notifications/feeds etc.
- The sole purpose of Accept() just to establish the connection between two machines (client and server) and carrying out TCP 3-way handshake (refer to your standard books)
- Accept() is called only on server side, and it returns the handle to the connection with client. Using this handle Server carries out all future communication with the client. This handle is called *Communication file descriptor*.



Socket Programming -> Accept() System Call

```
int comm_sock_fd = accept ( master_sock_tcp_fd,  
                           ( struct sockaddr *)&client_addr,  
                           &addr_len)
```



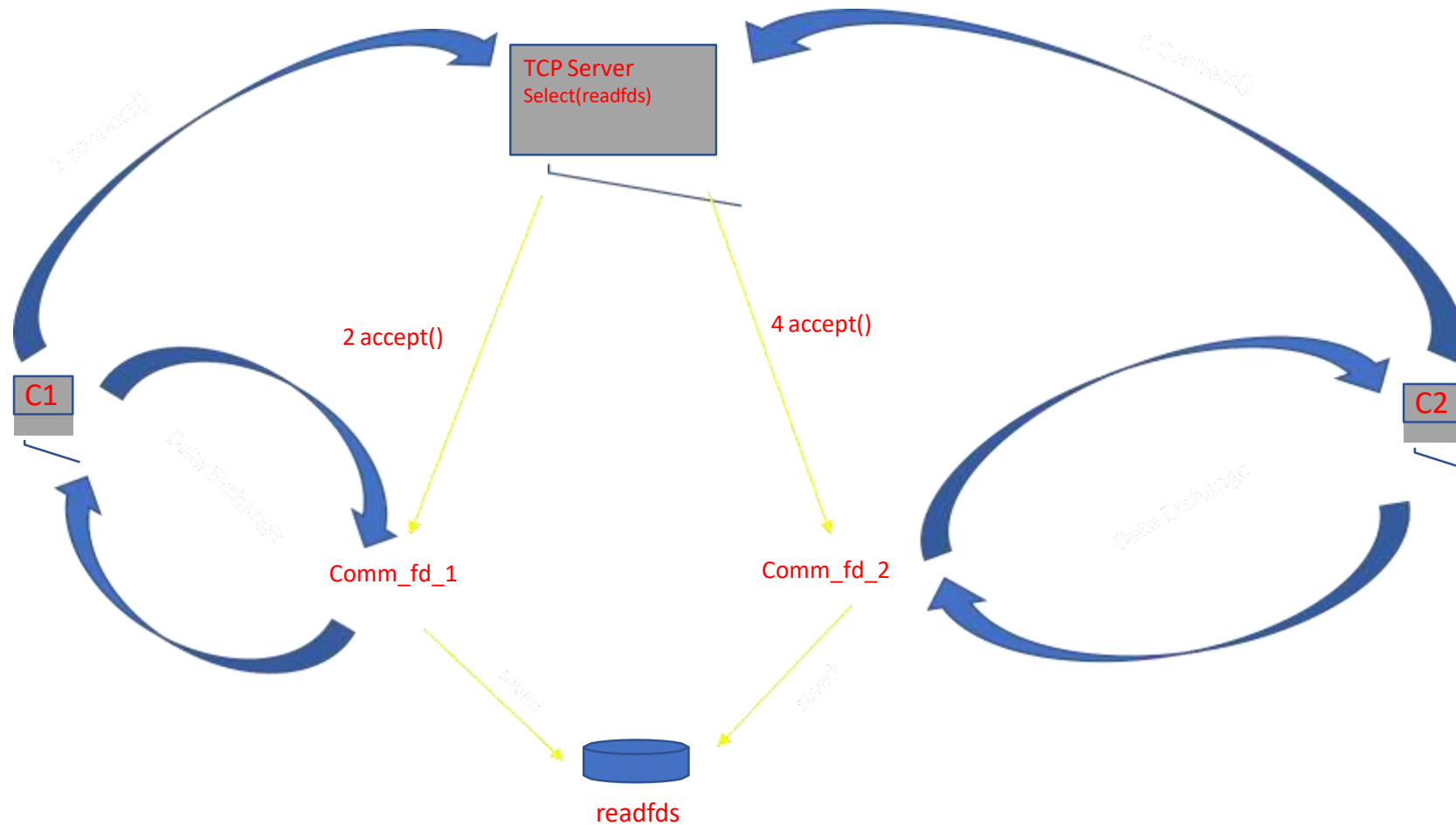
- Accept()
- Accept() returns a **file descriptor** (comm_socket_fd), which is just an integer, representing as the dedicated connection handle between connected client and server. Server carries out all communication with client using this returned integer value.
 - master_sock_tcp_fd is the fd of the socket which server has opened to listen to new client connection requests using *socket()*
 - client_addr is the structure which contains client info – ip address and tcp port no of client which has just connected
 - addr_len = size of predefined structure - *struct sockaddr* which is just a constant.
 - There are some more APIs – sendto and recvfrom, but there explicit discussion is not worth.
 - You will understand through code walk-through.

Socket Programming -> Select() System Call

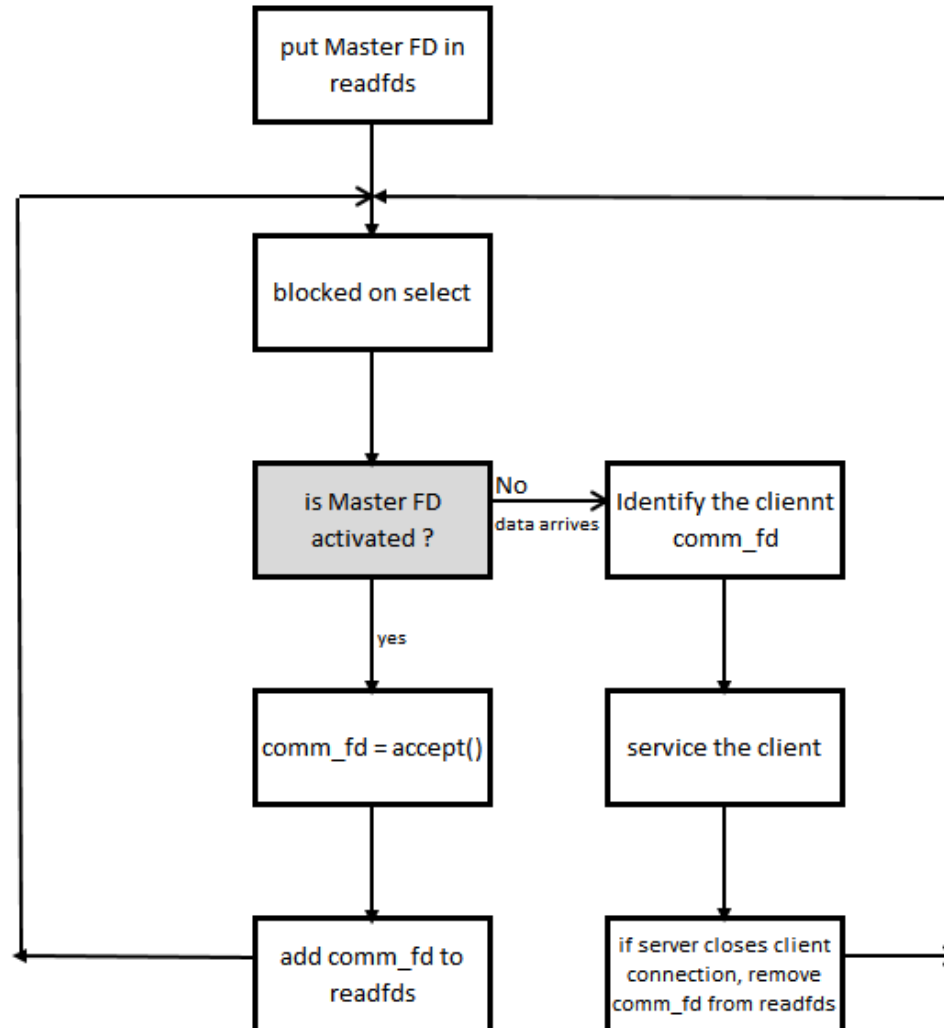
- Select() system calls allows the server machine to MONITOR multiple client connected connections and check which client has send the data to process
 - >> *Just like the class monitor keep an eye on all the students of the class at the same time*
- Select() system call is **blocking** System call – meaning, when it is executed, the code execution halts (similar to scanf/getch)
- Select() unblocks when either of two things happen on server side :
 - New connection request from new client arrives
 - Data request from existing connected client arrives
- When select() unblocks, server needs to check whether it's a new connection request Or new data request on existing connection. In the later case, server need to find which client has send the data.
- When server starts, the first thing it does is to create a master socket – A socket to detect arrival of new connection request from new client
- C provides a Data structure called *fd_set* which is a collection (or a set) of file descriptors `fd_set readfds;`

TCP/IP & UDP

Socket Programming -> Accept() and Select() System Call in operation



Socket Programming -> State machine Diagram of a server



Socket Programming -> Server Implementation

Code Walk

1. Initialize variables
2. Create Master socket
3. Bind
4. Listen
5. Initialize and fill readfds
6. Select
7. Accept the connection
8. service the client requests
9. close the connection
10. Goto 5

Socket Programming -> Server Implementation

Observations

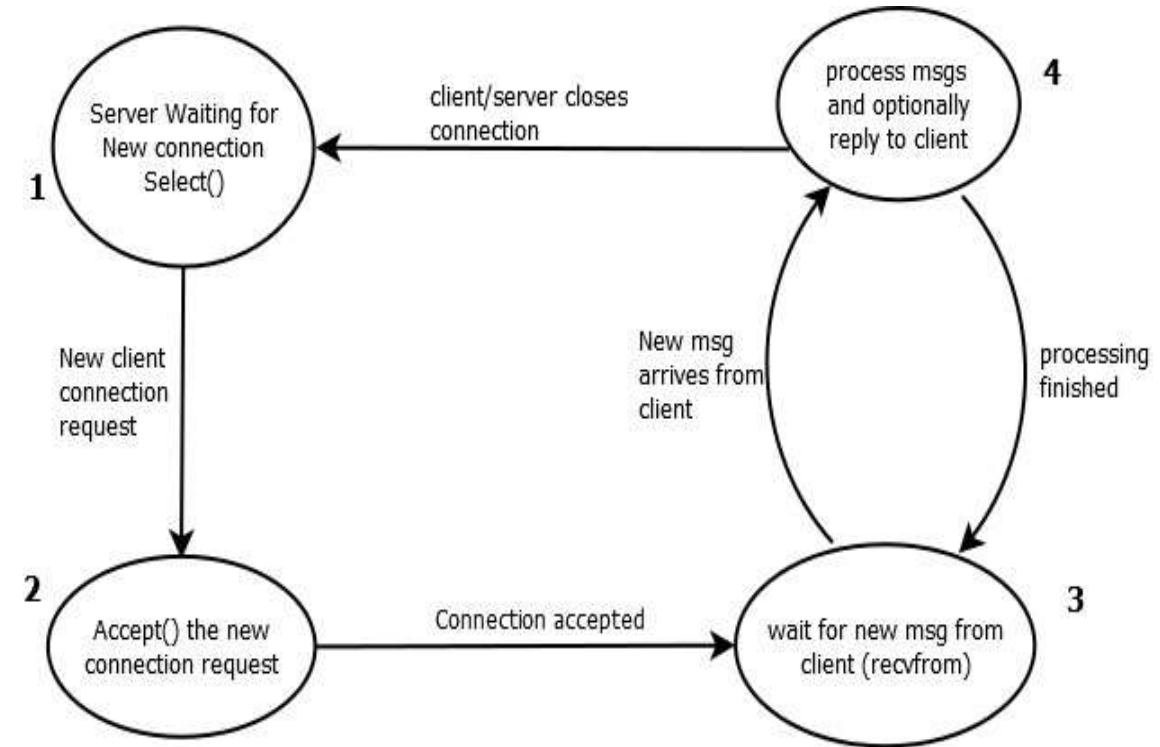
Server cannot accept other client's request while it is processing the current client (oscillating between stages 3 and 4).

Only one client can be served at a time.

This kind of design is used where there is only and only one dedicated client of server. Such server processes are used in system soft-wares where there is only one client and one server.

This server design cannot handle multiple clients connections

Remember, Servers are just another process which accepts some data from another process , nothing special.

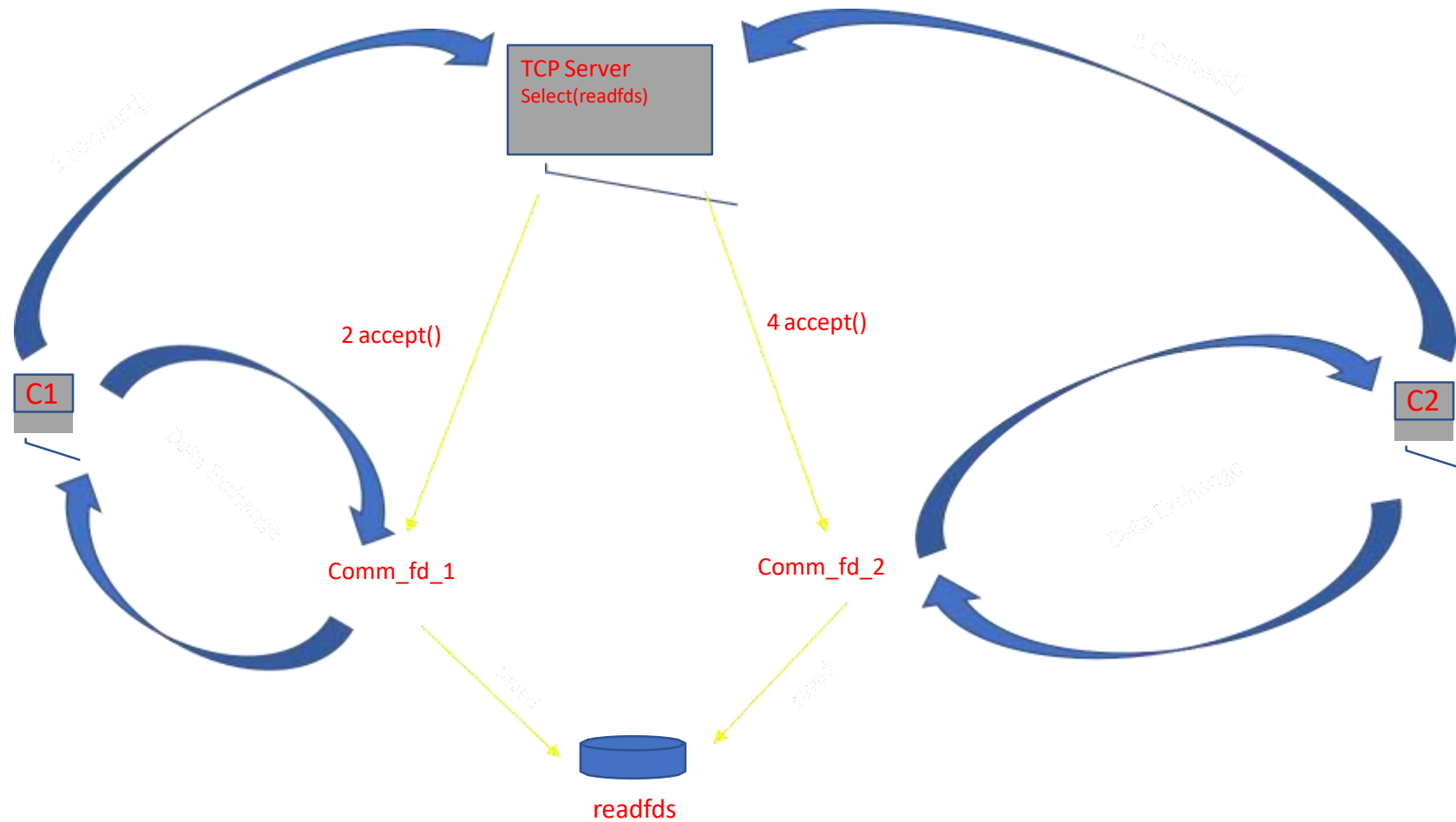


Socket Programming -> Multiplexed TCP Server Implementation

Code Walk For TCP Server which can handle multiple client connections

1. Initialize variables
2. Create Master socket
3. Bind
4. Listen
5. Initialize and fill readfds
6. Select
7. Accept the connection
8. service the client requests
9. close the connection
10. Goto 5

Socket Programming -> TCP Server with Multiplexing



Socket Programming -> Server Implementation

Socket programming APIs

1. `socket()` – Used to create a TCP/UDP master socket. Used on server and Client side
2. `select()` – Used for monitoring multiple file descriptors. Can be used on both client and server sides. Blocking system call. Blocks until new connection request Or data arrived on FDs present in readfds set.
3. `accept()` – Used on TCP server side. Used to accept the client connection request and complete 3-way handshake with client
4. `bind()` – Used on TCP/UDP server side. Used by the Server application process to inform operating system the criteria of packets of interests
5. `listen()` – Used on TCP/UDP server side. Used by the Server application process to inform operating system the length of Queue of incoming connection request/data request from clients
6. `recvfrom()` – Used on the TCP/UDP server and client side. Used by the Server/client process to read the data arrived on communication file descriptors. This call is blocking call by default. Process block if data hasn't arrived yet.
7. `sendto()` – Used to send data to client/server. Used on Server and client sides.
8. `close()` – Used to close the connection. Used by both – clients and Server

Socket Programming -> Select() Accept() code walk through

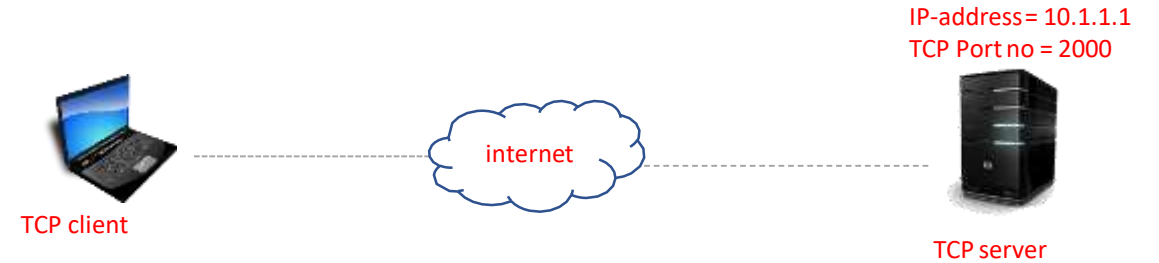
- Observations
 - Server Can accept and maintain upto 31 client's connection request (1 being Master skt fd)
 - Server can service one client at a time, however can maintain 31 connections
- If multiple client's data request comes together (upto n), all client will be serviced but one by one, where n is the value specified in listen()
 - With the increase in no of clients, clients can begin experience latency as they are serviced one by one
 - Thus, this server design is way better than previous one, but has scalability limitations
 - Scalability limitation will always be there, no matter how you design the server. To handle more no of clients, we need to deploy more independent server machines. That is why Facebook, Google have data centers with thousands of servers to handle millions of client request.
- So don't expect a single server machine to handle significant no of clients. Such a server is not limited by software design, but by hardware limitations.

Socket Programming -> TCP Client Implementation

Code Walk For TCP client

1. Initialize variables
2. Specify Server Credentials
3. Create a Communication socket
4. Connect with Server (Send connection initiation request to server)
5. Send data to Server
6. Receive response
7. (optional) close the connection
8. Goto 5 if wants to communicate more with the server over same connection

- No port number assignment by the programmer
- No bind() , no fd_set, no accept()



Socket Programming -> UDP Programming

- How to create UDP server machine and UDP client machine !
- If you have learnt creating TCP server machine, UDP programming is way much simpler than TCP programming ...
- Homework : Write your UDP Server and UDP client. Verrrryyy easy As compared to TCP counterparts. Infinite material available all over internet. Google...

Thank You



Developer
Wiki





Attribution 4.0 International (CC BY 4.0)

This is a human-readable summary of (and not a substitute for) the [license](#). [Disclaimer](#).

You are free to:

Share — copy and redistribute the material in any medium or format

Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

